

List of all functionality

1. Registration and Login feature

- a. **Register and Login:** The website will have a built-in registration feature allowing new users to sign up by providing a username, email, and password. A confirmation email will be sent to verify the user's email address.

2. Music Upload

- a. **Upload:** Registered users shall have the capability to upload music files to the website. Each upload must include the music name, the music image, and a short description written by the user.
- b. **Delete:** The users can also delete their own music files they have uploaded.
- c. **Supplemental info:** The website will only use music files that are in the public domain or created by the users themselves. Users also should not upload anything that may cause a copyright violation.

3. Music Browse

- a. **Main Page:** The root index of the current app will display three rows: the first row is a search bar, the second row is the recent uploads by users, the third row is a subset of all the songs.
- b. **All Songs:** The user can also enter the all songs page, which collects all the songs in a list. There is a filter that can sort the songs by their name, upload time, length, number of favorites, or only display my favorite songs.

4. Search

- a. **Main Page Search:** On the main page, users can type in the search bar and click search to do the query. The query results will include three rows. The first row is music names/descriptions that contains the search query. The second row is user names that match the query. The third row is playlists that contain the search query.
- b. **All Songs Search:** On the All Songs page, there will be a search bar implemented using Ajax for the user to do the query for the music names only.
- c. **Supplemental info:** a search engine should be implemented here to guarantee the user experience. However, here we will just use techniques like naive string matching to get the results.

5. Detailed Pages

- a. **Music Details:** The user can access the details of each song. The song details will include
 - i. The song name, description, the image, the upload user, length, and upload time
 - ii. Number of favorites and not favorites
 - iii. Comments to that song
- b. **User Details:** The user can access the details of the queried users. The user details will include
 - i. User name
 - ii. User avatar
 - iii. User description
 - iv. User registration time

- v. Number of favorites and not favorites that the user's uploaded songs has in total
 - vi. The list of songs the user has uploaded
- c. **Playlist Details:** The user can access the details of the queried playlists.
 - i. The creator of the playlist
 - ii. The creation date
 - iii. The list of songs in that playlist
 - iv. The sum of the number of favorites and not favorites the songs in the playlists has
- 6. **Music Play**
 - a. **Play:** Users can play the track they select.
 - b. **Minimize:** There is always a bar on the foot of the page, in which the current song being played will be displayed.
- 7. **Music Playlist**
 - a. **Create Playlist:** Users can create playlists, name their playlists, and add/delete music files to/from the playlists.
 - b. **My Favorites:** "My Favorites" is a special playlist. For each song, there is an "add to favorites" button that users can add/delete tracks to/from their favorites.
 - c. **Recent:** "Recent" is also a special playlist that will keep track of the recent 100 songs the users played.
 - d. **Sorting:** Inside each playlist, users can sort tracks by default (the time the track was added to the playlist), the track name, and the artist name.
- 8. **Listen Together**
 - a. Users can create 'Listen Together' rooms. The room id will be displayed in a section on the main page and other users can join or leave freely. The creator of the room selects a song to play, and all joined users will listen to the track in real-time synchronization, ensuring everyone is at the same point in the song.

First sprint backlog

Controller:

1. **Registration and Login feature (Chenglin Zhang)**
 - a. **Register and Login:** The website will have a built-in registration feature allowing new users to sign up by providing a username, email, and password. A confirmation email will be sent to verify the user's email address.
2. **Music Upload (Fulun Ma)**
 - a. **Upload:** Registered users shall have the capability to upload music files to the website. Each upload must include the music name, the music image, and a short description written by the user.
 - b. **Delete:** The users can also delete their own music files they have uploaded.
3. **Music Browse (Kunle Li)**

- a. **Main Page:** The root index of the current app will display three rows: the first row is a search bar, the second row is the recent uploads by users, the third row is the songs with the most favorites sorted in descending order.

Name and Andrew ID of the product owner

Fulun Ma, fulunm

Model:

1. Model Design and Implementation

a. User Model

Django comes with a built-in User model that typically covers registration and login functionalities. It is sufficient for basic authentication requirements.

b. User Profile Model

While Django's built-in User model handles authentication, we need to store additional information about each user.

Python

```
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='profile')
    avatar = models.ImageField(upload_to='user_avatars/', null=True,
blank=True)
    description = models.TextField(null=True, blank=True)
    registration_time = models.DateTimeField(auto_now_add=True)
    total_favorites = models.IntegerField(default=0)
    total_not_favorites = models.IntegerField(default=0)
```

c. Music Model

Represents individual music files uploaded by users. Each music is linked to a user and contains details like name, description, and the actual music file.

Python

```
class Music(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='uploaded_music')
    name = models.CharField(max_length=255)
    image = models.ImageField(upload_to='music_images/')
    description = models.TextField()
    file = models.FileField(upload_to='music_files/')
    upload_date = models.DateTimeField(auto_now_add=True)
    length = models.IntegerField(help_text="Length of the song in seconds")
```

```
favorites_count = models.IntegerField(default=0)
not_favorites_count = models.IntegerField(default=0)
```

d. **Music Comments**

To enhance social interaction on our platform, we add models for comments on music tracks:

Python

```
class MusicComment(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='music_comments')
    music = models.ForeignKey(Music, on_delete=models.CASCADE,
related_name='comments')
    comment = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

e. **Favorite/Not favorite Model**

To track favorites and not favorites for songs:

Python

```
class MusicFavorite(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='liked_music')
    music = models.ForeignKey(Music, on_delete=models.CASCADE, related_name='likes')
    like = models.BooleanField(default=True) # True for favorite, False for not
favorite

class Meta:
    unique_together = ('user', 'music')
```

f. **Playlist**

Users can create multiple playlists, which can contain many music files. The `is_favorites` and `is_recent` flags distinguish special playlists like "My Favorites" and "Recent."

Python

```
class Playlist(models.Model):
```

```

name = models.CharField(max_length=255)
user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='playlists')
musics = models.ManyToManyField(Music, related_name='playlists')
created_at = models.DateTimeField(auto_now_add=True)
is_favorites = models.BooleanField(default=False) # To distinguish "My
Favorites"
is_recent = models.BooleanField(default=False) # To distinguish "Recent"
playlist

```

g. ListenTogetherRoom Model

A feature allowing users to listen to music together in real-time. Each room has a creator, a current music, and participants.

```

Python
class ListenTogetherRoom(models.Model):
    creator = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='listen_together_rooms')
    current_music = models.ForeignKey(Music, on_delete=models.SET_NULL, null=True,
blank=True)
    room_id = models.CharField(max_length=255, unique=True)
    participants = models.ManyToManyField(User,
related_name='joined_listen_together_rooms')
    created_at = models.DateTimeField(auto_now_add=True)

```

h. Email Confirmation Model

For handling email confirmations during user registration, we need a model to store confirmation tokens:

```

Python
from django.utils.timezone import now

class EmailConfirmation(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    confirmation_token = models.CharField(max_length=100)
    created_at = models.DateTimeField(default=now)

    def is_expired(self):
        return (now() - self.created_at).days > 1 # Example: 1 day to expire

```

View:
Main page

User Name

Search music...

Recently Uploaded Songs

Recent Song 1

Artist 1

♥

▶

All Songs

Show Favorite Songs

Song 1

Artist 1

♥

▶

Song 2

Artist 2

♥

▶

Song 3

Artist 3

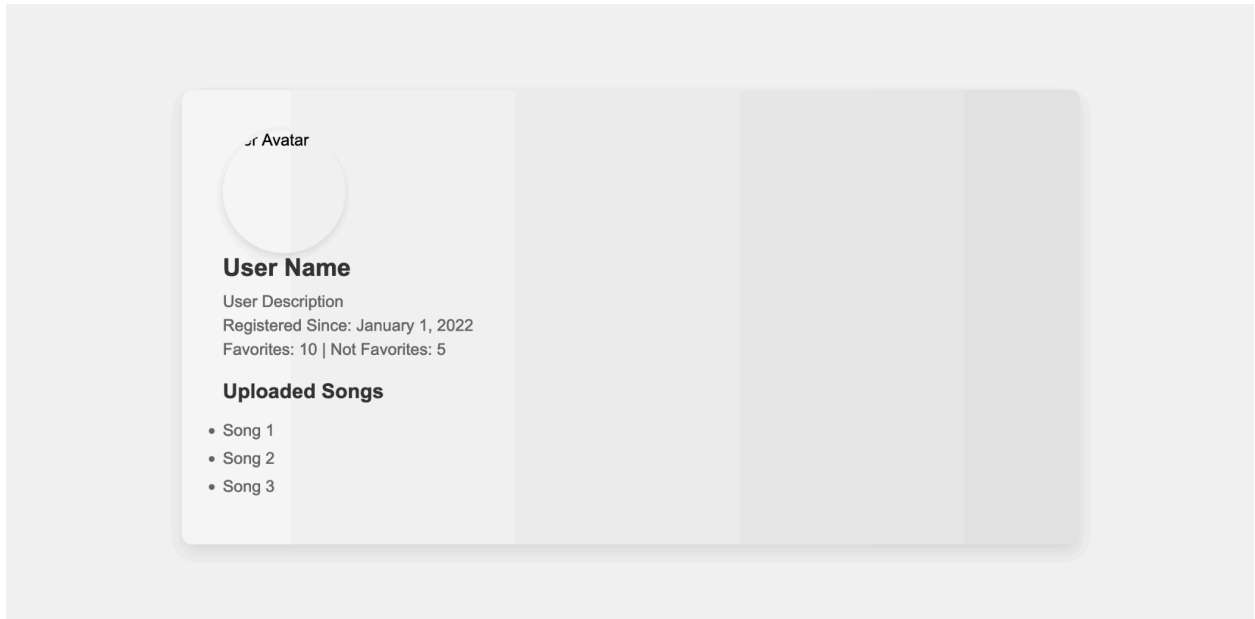
♥

▶

Now Playing: Song Name - Artist

00:00 / 00:00

User profile page



Music detail page

